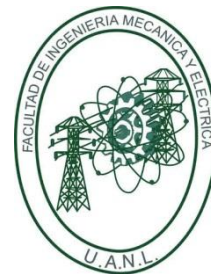




Universidad Autónoma de Nuevo León
Facultad de Ingeniería Mecánica y Eléctrica



*Controladores y Microcontroladores
Programables*

*Actividad Fundamental N.º05 –
Implementación de diferentes estructuras de
programación de un microcontrolador*

N.º de Equipo: 4

Brigada: 001

Nombre:	Matricula:	Numero de lista:
Heidi Pamela Martínez Martínez	1952947	6
Steven Neftalí Verdugo Figueroa	2132366	60
Ángel Armando Solís Jiménez	2132088	55
Carlos Emiliano Mendoza Ramírez	2048186	35

Nombre del profesor: Jesús Daniel Garza Camarena

Semestre Agosto – Diciembre 2025

Días de la clase y Hora: JueM4M6

San Nicolás de los Garza, N.L.

Fecha: 13/11/2025

COMPETENCIA ESPECÍFICA

Desarrollar habilidades en la programación avanzada de microcontroladores, enfocándose en el manejo de periféricos como interrupciones externas e interrupciones por timer, para diseñar sistemas embebidos capaces de gestionar eventos externos de manera asíncrona y controlar procesos temporizados con precisión. Esta competencia permitirá a los estudiantes implementar soluciones eficientes para el monitoreo y control de dispositivos electrónicos, optimizando el rendimiento y la respuesta del sistema en tiempo real.

INTRODUCCIÓN

El desarrollo de sistemas embebidos de alto rendimiento exige que los microcontroladores (MCU) sean capaces de responder de manera eficiente e inmediata a una variedad de eventos, tanto internos como externos. En este contexto, la programación basada en interrupciones y el manejo preciso del tiempo son pilares fundamentales que distinguen un diseño robusto y escalable de una implementación secuencial e ineficiente.

La incorporación de interrupciones externas permite al microcontrolador migrar desde un esquema de verificación constante (polling) a un modelo de respuesta asíncrona. En lugar de consumir ciclos de CPU preguntando repetidamente por el estado de un periférico o un botón, el sistema puede dedicarse a otras tareas, reaccionando instantáneamente solo cuando se genera un evento crítico. Esta capacidad no solo minimiza la latencia en la respuesta a eventos externos, sino que también optimiza significativamente el uso del procesador, un recurso limitado en cualquier sistema embebido.

Simultáneamente, el manejo del tiempo se vuelve vital para tareas que requieren precisión, como la multiplexación de displays o la medición y control de procesos. El uso estratégico de interrupciones por timer permite generar bases de tiempo exactas y repetibles, garantizando que operaciones periódicas, como la actualización de un contador regresivo o la gestión de la alarma, se realicen sin necesidad de utilizar funciones de espera bloqueantes (delay). Esta técnica es indispensable para mantener la capacidad de respuesta en tiempo real del sistema.

El presente proyecto se enfoca en aplicar estas competencias avanzadas mediante el diseño de un cronómetro regresivo. Al integrar interrupciones externas para la interacción con los botones de control, y timers para la lógica de conteo y el manejo del display multiplexado, el sistema no solo cumple con su función, sino que se convierte en un caso práctico del diseño moderno de sistemas embebidos, destacando la sinergia entre la gestión de eventos asíncronos y la precisión temporal, todo ello orquestado por una máquina de estados finitos.

Implementación de diferentes estructuras de programación de un microcontrolador

DESCRIPCIÓN DE LA ACTIVIDAD:

Investigación y Definiciones

Define los siguientes términos clave relacionados con los periféricos internos y externos y su papel en la programación de sistemas embebidos. Utiliza fuentes confiables y cita en formato APA.

- Librerías para Drivers
- Debouncing (Eliminación de Rebotes)
- Interrupciones Externas en microcontroladores
- Interrupciones por Timer en microcontroladores
- Máquina de estados finitos en microcontroladores

Comparación de Métodos de Programación

Describe las diferencias entre la programación secuencial y polling vs la programación basada en interrupciones.

Explica:

- **Programación Secuencial:** Cómo sigue una secuencia fija de instrucciones y cuándo es útil.
- **Programación con Interrupciones:** Cómo permite que el microcontrolador reaccione a eventos externos de manera inmediata, mejorando la capacidad de respuesta.

Práctica

Desarrollar un cronómetro que inicie en 59:59, utilizando una máquina de estados para gestionar su funcionamiento. El cronómetro regresivo contará con dos botones, start/stop y configurar, ambos controlados por interrupciones externas, para ajustar y controlar el tiempo en un display multiplexado de 4 dígitos manejado únicamente por timers no delays. Se debe de usar transistores en la implementación física para el control del display y el buzzer de la alarma, en la simulación no es necesario.

1. Configuración del Tiempo (puedes hacer que los minutos duren menos para probar tu proyecto):
 - Al presionar el botón configurar por primera vez, botón start/stop ajustará los segundos.
 - Al presionar el botón configurar nuevamente, el botón start/stop permitirá ajustar los minutos.
 - Una vez configurados los minutos y segundos, el sistema espera a que se presione start/stop para comenzar la cuenta regresiva.
2. Iniciar/Pausar:
 - El botón start/stop, controlado por una interrupción externa con un circuito antirrebotes por hardware, inicia o pausa el cronómetro según el estado actual de la máquina de estados.

3. Alarma Final:

- Cuando el cronómetro llega a 00:00, suena un buzzer controlado por un transistor.
- El buzzer se detiene al presionar cualquiera de los dos botones (configurar o start/stop), ambos controlados por interrupciones externas.

1. INVESTIGACIÓN Y DEFINICIONES

○ LIBRERÍAS PARA DRIVERS

Un driver (controlador) es un fragmento de código diseñado para permitir que el microcontrolador (MCU) interactúe con un periférico específico (hardware). Las librerías para drivers son colecciones de código que facilitan la comunicación entre el sistema operativo y el hardware. Permiten a los desarrolladores usar funciones predefinidas para controlar dispositivos, sin tener que escribir el código desde cero, lo que agiliza el desarrollo y asegura la reutilización del código. En esencia, son un conjunto de funciones y bloques que resuelven problemas específicos, como la comunicación con un dispositivo, permitiendo a los programas interactuar con el hardware de manera eficiente.

○ DEBOUNCING (ELIMINACIÓN DE REBOTES)

El "debouncing" es una práctica de programación utilizada para limitar la velocidad a la que se ejecuta una función. Asegura que una función solo se llame una vez dentro de un período de tiempo especificado, independientemente de cuántas veces se haya activado. Esto es particularmente útil en escenarios donde una acción se puede realizar varias veces en rápida sucesión, como los eventos de entrada del usuario.

○ INTERRUPTIONES EXTERNAS EN MICROCONTROLADORES

Las interrupciones externas sirven para detectar un estado lógico o un cambio de estado en alguna de las terminales de entrada de un microcontrolador, con su uso se evita un sondeo continuo en la terminal de interés. Son útiles para monitorear interruptores, botones o sensores con salida a relevador.

○ INTERRUPTIONES POR TIMER EN MICROCONTROLADORES

Las interrupciones son eventos que hacen que el microcontrolador PIC deje de realizar la tarea actual y pase a efectuar otra actividad. Al finalizar la segunda actividad retorna a la primera y continúa a partir del punto donde se produjo la interrupción.

Una interrupción de temporizador es una interrupción dedicada que funciona como un reloj de alta precisión, contando y controlando eventos temporales con precisión de microsegundos. Los temporizadores de hardware integrados en el ESP32 se utilizan con dos propósitos: proporcionar retardos de tiempo precisos y actuar como contadores en algunas aplicaciones.

○ MÁQUINA DE ESTADOS FINITOS EN MICROCONTROLADORES

Las máquinas de estados finitos (FSM) en microcontroladores son un modelo abstracto para diseñar sistemas que reaccionan a eventos. Describen un sistema con un número finito de estados y cómo transita entre ellos basándose en entradas, produciendo salidas en el proceso. Son una herramienta fundamental para gestionar la complejidad de tareas, permitiendo que un microcontrolador ejecute una serie de acciones complejas de manera estructurada y predecible, controlando por ejemplo actuadores en respuesta a señales de sensores.

2. COMPARACIÓN DE MÉTODOS DE PROGRAMACIÓN

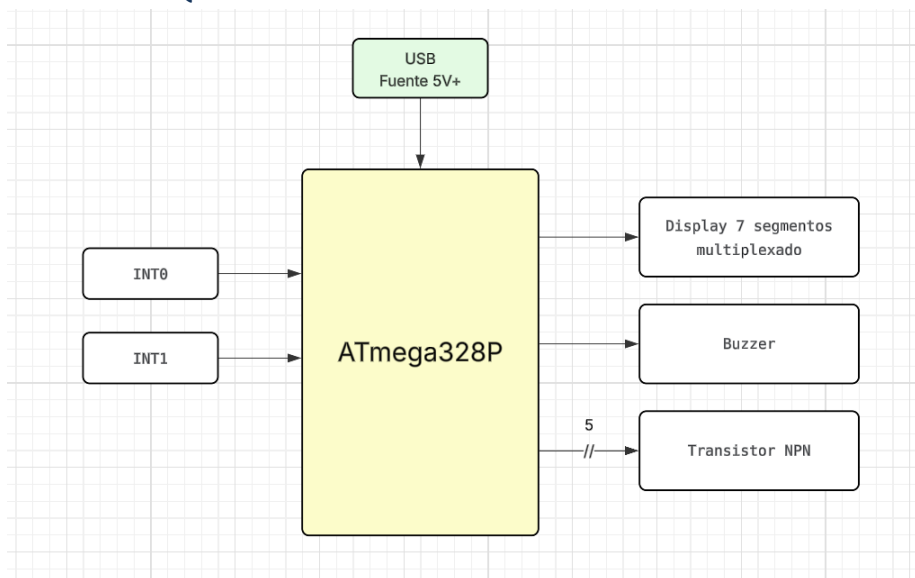
A continuación, se presenta una tabla comparativa detallada la programación secuencial y polling vs la programación basada en interrupciones. Se destacan aspectos clave como su flujo de control, respuesta a eventos, latencia, uso de CPU, complejidad y aplicaciones típicas.

Característica	Programación Secuencial / Polling	Programación Basada en Interrupciones
Flujo de Control	Fijo, lineal. El programa ejecuta instrucciones una tras otra.	Impulsado por eventos. El flujo normal se detiene temporalmente ante un evento (interrupción).
Respuesta a Eventos	Lenta o diferida. Debe esperar a que se ejecute la instrucción de "polling" (chequeo) en el bucle principal.	Inmediata. El microcontrolador detiene su tarea actual para atender el evento apenas ocurre.
Latencia	Alta. El tiempo de respuesta depende de la duración del ciclo de ejecución principal.	Baja y predecible. El tiempo de respuesta es muy rápido, crucial para sistemas de tiempo real.
Uso de CPU	Ineficiente/Alto. La CPU gasta tiempo valioso verificando constantemente (<i>polling</i>).	Eficiente/Bajo. La CPU solo se activa para atender el evento cuando es notificada por el <i>hardware</i> .
Complejidad	Generalmente más simple y fácil de depurar para tareas sencillas.	Más complejo por la necesidad de gestionar el contexto (guardar y restaurar el estado de la CPU) y la concurrencia.
Aplicaciones Típicas	Tareas repetitivas y predecibles, sistemas simples sin requisitos estrictos de tiempo real.	Sistemas de tiempo real, comunicación serial, detección de botones/sensores rápidos, manejo eficiente de periféricos.

3. LISTA DE MATERIALES

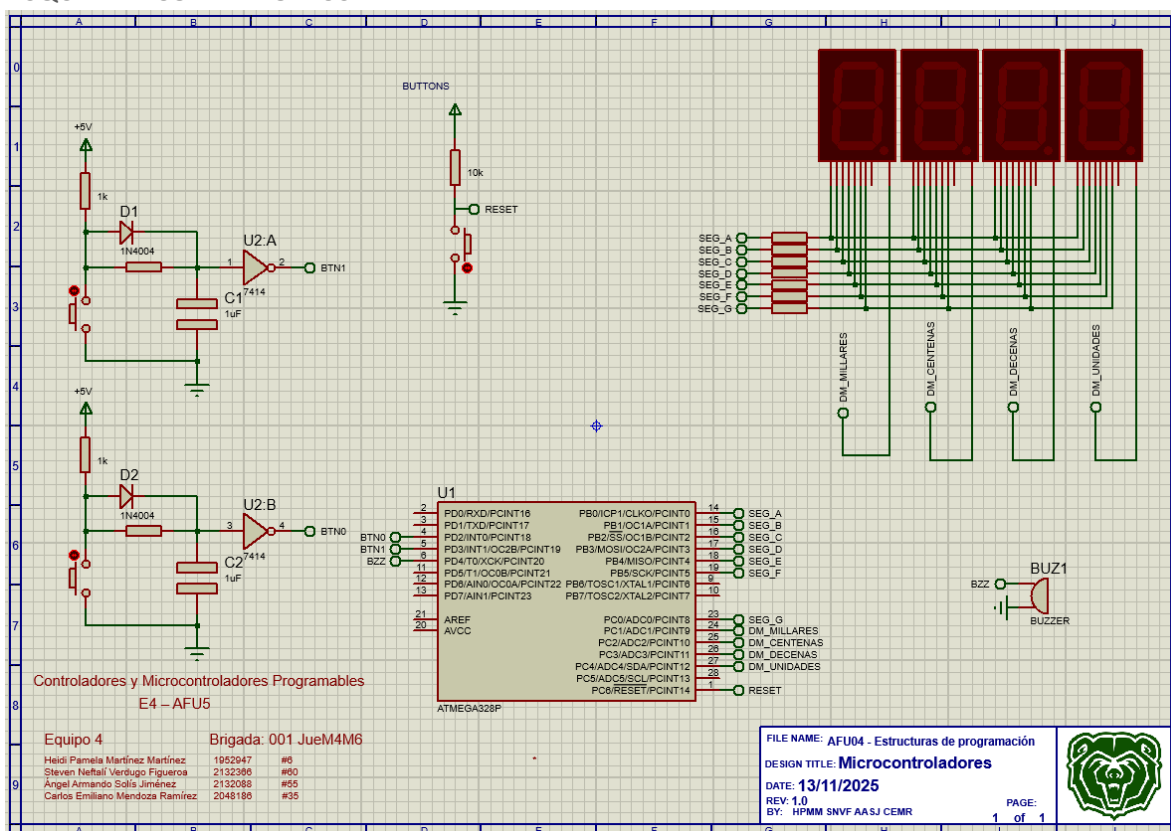
Material	Unidades	Descripción / Especificación Técnica	Función en el Sistema
Arduino R3	1	Microcontrolador ATmega328P, 14 pines digitales, 6 entradas analógicas, funciona a 5V, compatibilidad con USB y alimentación externa.	Actúa como el cerebro del sistema, ejecuta el programa, controla entradas y salidas, gestiona sensores, actuadores y la lógica principal.
Protoboard	1	Tablero de pruebas sin soldadura, múltiples filas y columnas para conexiones rápidas, compatible con componentes electrónicos estándar.	Permite montar el circuito sin soldar, realizar pruebas, conectar componentes y prototipar el diseño electrónico del sistema.
Resistencia 1k Ω	5	Tolerancia 5%, 1/4 W.	Pull-down para los botones o configuración de entradas.
Resistencia 220 Ω	9	Tolerancia 5%, 1/4 W.	Limitan la corriente de los segmentos del display.
Display de 7 segmentos (4 dígitos)	1	Cátodo común / Ánodo común (según diseño), tensión 5 V.	Muestra el tiempo del cronómetro mediante multiplexación.
Jumpers	19	Cables de conexión tipo macho-macho, macho-hembra o hembra-hembra; longitudes variables; recubiertos de PVC.	Cables de conexión tipo macho-macho, macho-hembra o hembra-hembra; longitudes variables; recubiertos de PVC.
Transistores NPN (BC547-B)	5	Ganancia hFE $\approx$ 100, corriente máx. 100 mA.	Controlan la activación de cada dígito del display y el buzzer.
Botón	2	Tensión 5 V, rebote mecánico 10 ms.	Controlan funciones de Start/Stop y Configurar.
Buzzer	1	5 V, consumo <30 mA.	Emite la señal acústica cuando el tiempo llega a 00:00.

4. DIAGRAMA DE BLOQUES

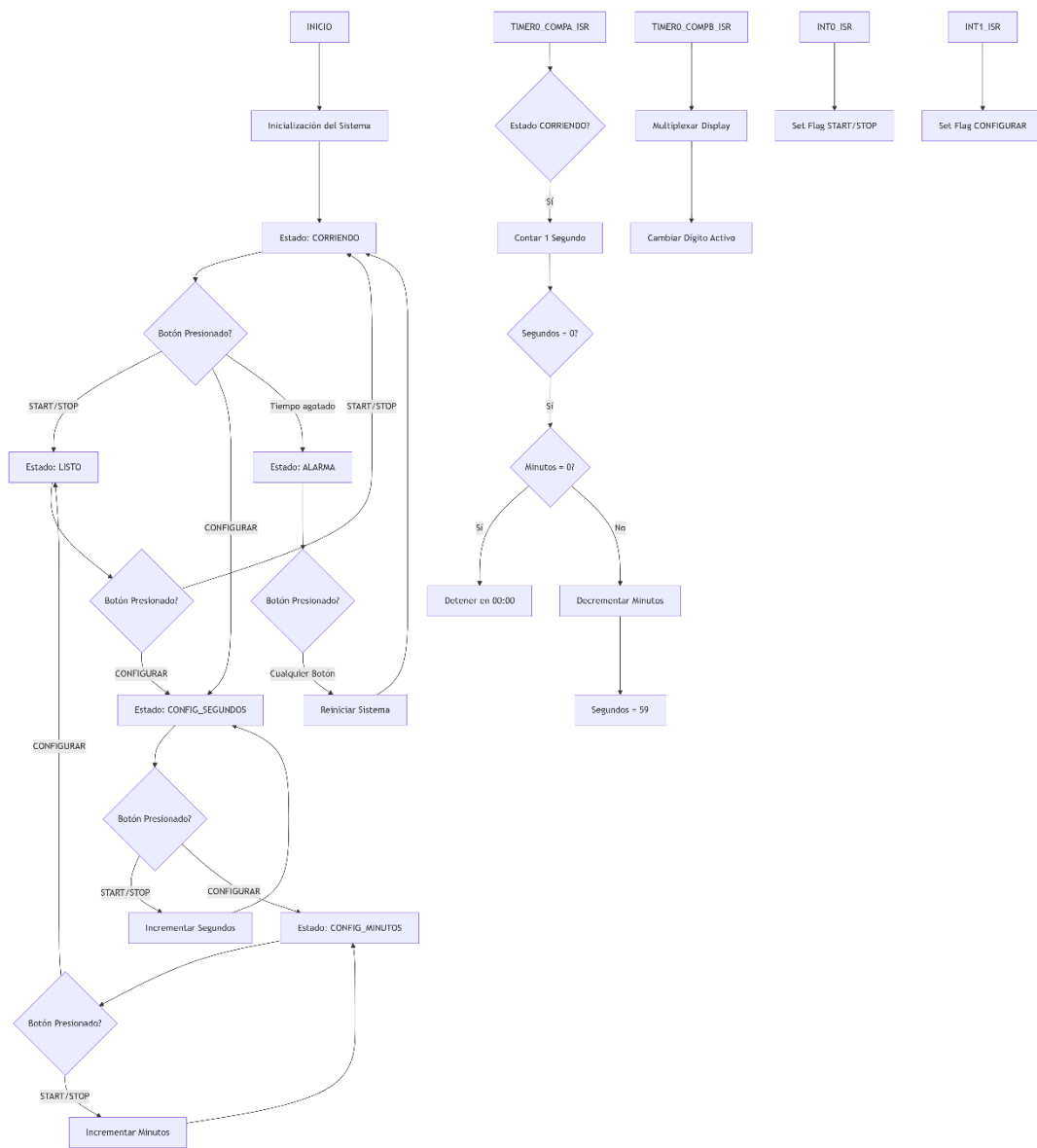


5. DIAGRAMA ESQUEMÁTICO EN EDA

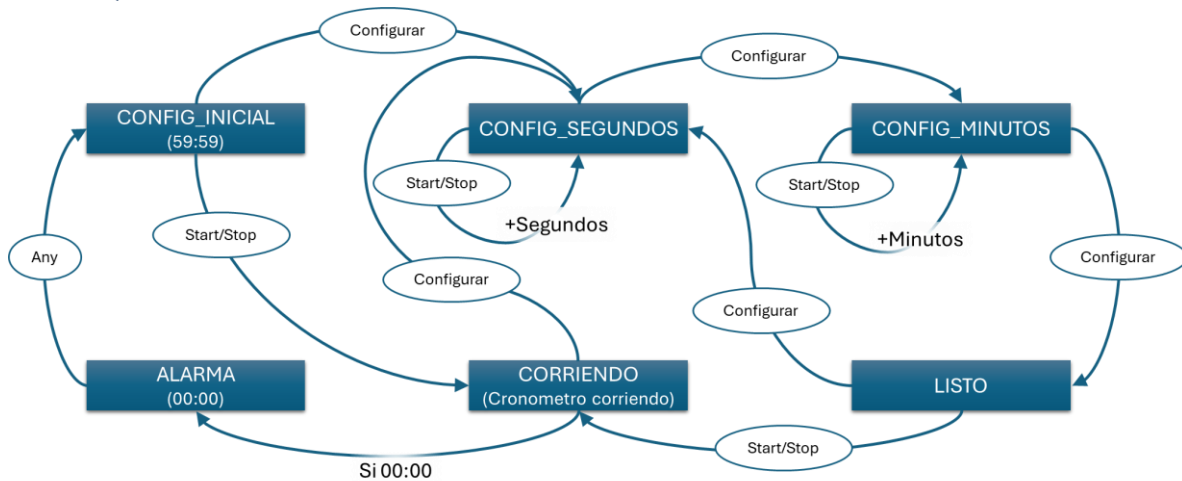
ESQUEMÁTICO EN PROTEUS



6. DIAGRAMA DE FLUJO EN ALTO NIVEL



7. MÁQUINA DE ESTADOS



En la máquina de estados encontramos 6 estados:

- **CONFIG_INICIAL:** Se activa al presionando cualquier botón después de que el sistema llegue al estado “ALARMA”, muestra el temporizador en 59:59.
- **CONFIG_SEGUNDOS:** Se activa al presionar el botón “Configurar” desde el estado “CONFIG_INICIAL”, “LISTO” o “CORRIENDO”. Al presionar el botón “Start/Stop” en este estado se incrementa un segundo al temporizador.
- **CONFIG_MINUTOS:** Se activa al presionar el botón “Configurar” desde el estado “CONFIG_SEGUNDOS”. Al presionar el botón “Start/Stop” en este estado se incrementa un minuto al temporizador.
- **LISTO:** Se activa al presionar el botón “Configurar” desde el estado “CONFIG_MINUTOS”. Indica que el temporizador está listo para comenzar.
- **CORRIENDO:** Se activa al iniciar el sistema con la configuración predeterminada “59:59” o al presionar el botón “Start/Stop” desde el estado “LISTO” con la configuración establecida por el usuario. Inicia el temporizador en una cuenta regresiva hasta llegar a “00:00”.
- **ALARMA:** Se activa cuando el temporizador llega a 00:00. Activa una alarma.

8. CÓDIGO EN LENGUAJE C O ASM NO ARDUINO

```
/*
 * Copyright (C) 2025 by Equipo 4
 *
 * Heidi Pamela Martínez Martínez          1952947  #6
 * Steven Neftalí Verdugo Figueroa        2132366  #60
 * Ángel Armando Solís Jiménez            2132088  #55
 * Carlos Emiliano Mendoza Ramírez        2048186  #35
 *
 *
 * Attribution-NonCommercial-ShareAlike 3.0(CC-BY-NC-SA 3.0)
 * AFU05 - Estructuras de programación
 * Dispositivo: atmega328p
 * Lenguaje C
 * Rev: 1.0
 */
```

```

*   Licencia:                                     *
*                                                                 Fecha: 12/11/25      *
*****/

#include <avr/io.h>
#define F_CPU 16000000UL //16 Mhz
#include <util/delay.h>
#include <avr/interrupt.h>

//Display de 7 segmentos
#define DISPLAY_7SEG_AF_DDRX  DDRB
#define DISPLAY_7SEG_AF_PORTX PORTB
#define DISPLAY_7SEG_G_DDRX   DDRC
#define DISPLAY_7SEG_G_PORTX  PORTC
#define DISPLAY_7SEG_SEGA     PORTB0
#define DISPLAY_7SEG_SEGB     PORTB1
#define DISPLAY_7SEG_SEGC     PORTB2
#define DISPLAY_7SEG_SEGD     PORTB3
#define DISPLAY_7SEG_SEGE     PORTB4
#define DISPLAY_7SEG_SEGF     PORTB5
#define DISPLAY_7SEG_SEGG     PORTC0

//Botones con interrupciones externas
#define INT0_DDRX  DDRD
#define INT0_PORTX PORTD
#define INT0_BTN0  PORTD2 // Botón START/STOP

#define INT1_DDRX  DDRD
#define INT1_PORTX PORTD
#define INT1_BTN1  PORTD3 // Botón CONFIGURAR

//Display Multiplexado
#define DM_DDRX    DDRC
#define DM_PORTX   PORTC
#define DM_MILLARES PORTC1
#define DM_CENTENAS PORTC2
#define DM_DECENAS  PORTC3
#define DM_UNIDADES PORTC4

//Buzzer
#define BUZZER_DDRX  DDRD
#define BUZZER_PORTX PORTD
#define BUZZER_PIN   PORTD4

//Índices de dígitos
#define MIN_DEC  0
#define MIN_UNI  1
#define SEG_DEC  2
#define SEG_UNI  3

// Arrays de dígitos y tiempo
volatile uint8_t timedigits[4];
volatile uint8_t minutos = 59;
volatile uint8_t segundos = 59;

// Array de números para display de 7 segmentos
uint8_t display_numbers_array[10] =

```

```

{ //Xgfedcba
    0b00111111, // 0
    0b00000110, // 1
    0b01011011, // 2
    0b01001111, // 3
    0b01100110, // 4
    0b01101101, // 5
    0b01111101, // 6
    0b00000111, // 7
    0b01111111, // 8
    0b01101111 // 9
};

// Variables de control
volatile uint8_t timer0_barrido_flag = 0; // Control de multiplexación
volatile uint16_t timer0_1seg_counter = 0; // Contador para 1 segundo
volatile uint8_t int0_debounce_flag = 0; // Antirrebote INT0
volatile uint8_t int1_debounce_flag = 0; // Antirrebote INT1
volatile uint16_t buzzer_counter = 0; // Contador para buzzer
volatile uint8_t buzzer_active = 0; // Estado del buzzer

// Estados de la máquina de estados
enum states {
    CONFIG_INICIAL, // Estado inicial
    CONFIG_SEGUNDOS, // Configurando segundos
    CONFIG_MINUTOS, // Configurando minutos
    LISTO, // Esperando a presionar start
    CORRIENDO, // Cronómetro corriendo
    ALARMA // Tiempo terminado, sonando alarma
} state;

// Prototipos de funciones
void display_init(void);
void display_show_number_array(uint8_t number);
void int0_init(void);
void int1_init(void);
void dm_init(void);
void dm_show_time(uint8_t mins, uint8_t segs);
void timer0_init(void);
void timer0_run(void);
void buzzer_init(void);
void buzzer_on(void);
void buzzer_off(void);
void update_time_digits(void);

///-----MAIN-----/////
int main(void)
{
    cli(); // Deshabilitar interrupciones

    // Inicialización
    display_init();
    int0_init();
    int1_init();
    dm_init();
    buzzer_init();
    timer0_init();
    timer0_run();

```

```

state = CORRIENDO; // Inicia directamente en cuenta regresiva
timer0_1seg_counter = 0; // Reiniciar contador

sei(); // Habilitar interrupciones

while (1)
{
    switch (state)
    {
        case CONFIG_INICIAL:
            // Este estado ya no se usa al inicio, solo al reiniciar
            // Mostrar tiempo inicial 59:59
            dm_show_time(minutos, segundos);

            // Transición: presionar CONFIGURAR para ajustar segundos
            if (int1_debounce_flag == 1)
            {
                state = CONFIG_SEGUNDOS;
                int1_debounce_flag = 0;
            }
            break;

        case CONFIG_SEGUNDOS:
            // Mostrar tiempo actual
            dm_show_time(minutos, segundos);

            // Transición: START/STOP incrementa segundos
            if (int0_debounce_flag == 1)
            {
                segundos++;
                if (segundos > 59) segundos = 0;
                int0_debounce_flag = 0;
            }

            // Transición: CONFIGURAR para ajustar minutos
            if (int1_debounce_flag == 1)
            {
                state = CONFIG_MINUTOS;
                int1_debounce_flag = 0;
            }
            break;

        case CONFIG_MINUTOS:
            // Mostrar tiempo actual
            dm_show_time(minutos, segundos);

            // Transición: START/STOP incrementa minutos
            if (int0_debounce_flag == 1)
            {
                minutos++;
                if (minutos > 59) minutos = 0;
                int0_debounce_flag = 0;
            }

            // Transición: CONFIGURAR para estar listo
            if (int1_debounce_flag == 1)
            {

```

```

        state = LISTO;
        int1_debounce_flag = 0;
    }
    break;

case LISTO:
    // Mostrar tiempo configurado
    dm_show_time(minutos, segundos);

    // Transición: START/STOP para comenzar
    if (int0_debounce_flag == 1)
    {
        state = CORRIENDO;
        timer0_1seg_counter = 0;
        int0_debounce_flag = 0;
    }

    // Permite reconfigurar
    if (int1_debounce_flag == 1)
    {
        state = CONFIG_SEGUNDOS;
        int1_debounce_flag = 0;
    }
    break;

case CORRIENDO:
    // Mostrar tiempo actual
    dm_show_time(minutos, segundos);

    // Transición: START/STOP para pausar
    if (int0_debounce_flag == 1)
    {
        state = LISTO;
        int0_debounce_flag = 0;
    }

    // Transición: CONFIGURAR para reconfigurar
    if (int1_debounce_flag == 1)
    {
        state = CONFIG_SEGUNDOS;
        int1_debounce_flag = 0;
    }

    // Verificar si llegó a 00:00
    if (minutos == 0 && segundos == 0)
    {
        state = ALARMA;
        buzzer_active = 1;
        buzzer_counter = 0;
    }
    break;

case ALARMA:
    // Mostrar 00:00
    dm_show_time(0, 0);

    // Controlar buzzer (encender/apagar cada 0.5 seg para
    efecto intermitente)

```

```

        if (buzzer_active)
        {
            if (buzzer_counter < 50) // 0.5 seg ON
            {
                buzzer_on();
            }
            else if (buzzer_counter < 100) // 0.5 seg OFF
            {
                buzzer_off();
            }
            else
            {
                buzzer_counter = 0; // Reiniciar ciclo
            }
        }

// Transición: cualquier botón detiene el buzzer y
reinicia el sistema
if (int0_debounce_flag == 1 || int1_debounce_flag == 1)
{
    buzzer_off();           // Apagar buzzer

    buzzer_active = 0;      // Desactivar bandera del
    buzzer

    minutos = 59;           // Reiniciar tiempo a 59:59
    segundos = 59;
    state = CORRIENDO;      // Volver a contar

    automáticamente
    timer0_1seg_counter = 0; // Reiniciar contador de
    segundos

    int0_debounce_flag = 0; // Limpiar banderas
    int1_debounce_flag = 0;

    break;
}
} //fin while
} //fin main

//Definiciones de funciones
void display_init(void)
{
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGA);
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGB);
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGC);
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGD);
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGE);
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGF);
    DISPLAY_7SEG_G_DDRX  |= (1<<DISPLAY_7SEG_SEGG);
}

void display_show_number_array(uint8_t number)
{
    if (number > 9) number = 0; // Protección

    DISPLAY_7SEG_AF_PORTX = (display_numbers_array[number] & 0b00111111) |
        (DISPLAY_7SEG_AF_PORTX & 0b11000000);
    DISPLAY_7SEG_G_PORTX = ((display_numbers_array[number] & 0b01000000) >> 6) |
        (DISPLAY_7SEG_G_PORTX & 0b11111110);
}

```

```

}

void int0_init(void)
{
    // Configurar botón como entrada
    INT0_DDRX &= ~(1<<INT0_BTN0);
    // Activar pull-up interno
    INT0_PORTX |= (1<<INT0_BTN0);

    // Configurar rising edge (flanco de subida)
    EICRA |= (1<<ISC01) | (1<<ISC00);

    // Habilitar interrupción
    EIMSK |= (1<<INT0);
}

void int1_init(void)
{
    // Configurar botón como entrada
    INT1_DDRX &= ~(1<<INT1_BTN1);
    // Activar pull-up interno
    INT1_PORTX |= (1<<INT1_BTN1);

    // Configurar rising edge (flanco de subida)
    EICRA |= (1<<ISC11) | (1<<ISC10);

    // Habilitar interrupción
    EIMSK |= (1<<INT1);
}

void dm_init(void)
{
    DM_DDRX |= (1<<DM_MILLARES) | (1<<DM_CENTENAS) | (1<<DM_DECENAS) |
(1<<DM_UNIDADES);
}

void update_time_digits(void)
{
    timedigits[MIN_DEC] = (minutos / 10);
    timedigits[MIN_UNI] = (minutos % 10);
    timedigits[SEG_DEC] = (segundos / 10);
    timedigits[SEG_UNI] = (segundos % 10);
}

void dm_show_time(uint8_t mins, uint8_t segs)
{
    // Actualizar dígitos
    timedigits[MIN_DEC] = (mins / 10);
    timedigits[MIN_UNI] = (mins % 10);
    timedigits[SEG_DEC] = (segs / 10);
    timedigits[SEG_UNI] = (segs % 10);

    // Multiplexación del display
    switch (timer0_barrido_flag)
    {
        case MIN_DEC:
            DM_PORTX = (1<<DM_MILLARES) | (DM_PORTX & 0b11100001);
            display_show_number_array(timedigits[MIN_DEC]);

```



```

        break;
    case MIN_UNI:
        DM_PORTX = (1<<DM_CENTENAS) | (DM_PORTX & 0b11100001);
        display_show_number_array(timedigits[MIN_UNI]);
        break;
    case SEG_DEC:
        DM_PORTX = (1<<DM_DECENAS) | (DM_PORTX & 0b11100001);
        display_show_number_array(timedigits[SEG_DEC]);
        break;
    case SEG_UNI:
        DM_PORTX = (1<<DM_UNIDADES) | (DM_PORTX & 0b11100001);
        display_show_number_array(timedigits[SEG_UNI]);
        break;
    }
}

void buzzer_init(void)
{
    // Configurar pin del buzzer como salida
    BUZZER_DDRX |= (1<<BUZZER_PIN);
    buzzer_off();
}

void buzzer_on(void)
{
    BUZZER_PORTX |= (1<<BUZZER_PIN);
}

void buzzer_off(void)
{
    BUZZER_PORTX &= ~(1<<BUZZER_PIN);
}

void timer0_init(void)
{
    // MODO CTC
    TCCR0A &= ~(1<<WGM00);
    TCCR0A |= (1<<WGM01);
    TCCR0B &= ~(1<<WGM02);

    // TOP para ~0.01 segundos con prescaler 1024
    OCR0A = 156;
    OCR0B = 156;

    // Habilitar interrupciones
    TIMSK0 |= (1<<OCIE0A); // Para cuenta regresiva de 1 segundo
    TIMSK0 |= (1<<OCIE0B); // Para multiplexación del display
}

void timer0_run(void)
{
    // Reiniciar contador
    TCNT0 = 0;

    // Prescaler 1024
    TCCR0B |= (1<<CS02) | (1<<CS00);
    TCCR0B &= ~(1<<CS01);
}

```

```

// ISR Timer0 Compare A - Control de cuenta regresiva (cada ~0.01 seg)
ISR(TIMER0_COMPA_vect)
{
    // Incrementar contador para buzzer
    if (buzzer_active)
    {
        buzzer_counter++;
    }

    // Solo decrementar si está en estado CORRIENDO
    if (state == CORRIENDO)
    {
        timer0_1seg_counter++;

        if (timer0_1seg_counter >= 100) // 100 x 0.01 seg = 1 segundo
        {
            timer0_1seg_counter = 0;

            // Decrementar segundos
            if (segundos > 0)
            {
                segundos--;
            }
            else
            {
                segundos = 59;
                if (minutos > 0)
                {
                    minutos--;
                }
                else
                {
                    // Llegó a 00:00
                    segundos = 0;
                    minutos = 0;
                }
            }
        }
    }
}

// ISR Timer0 Compare B - Multiplexación del display (cada ~0.01 seg)
ISR(TIMER0_COMPB_vect)
{
    timer0_barrido_flag++;
    if (timer0_barrido_flag >= 4)
    {
        timer0_barrido_flag = 0;
    }
}

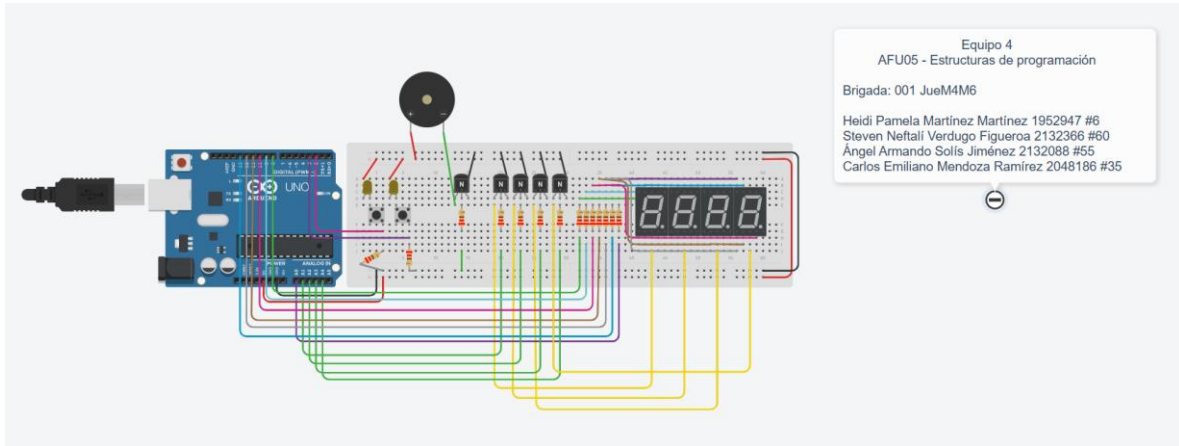
// ISR INT0 - Botón START/STOP
ISR(INT0_vect)
{
    _delay_ms(50); // Antirrebote simple
    int0_debounce_flag = 1;
}

```

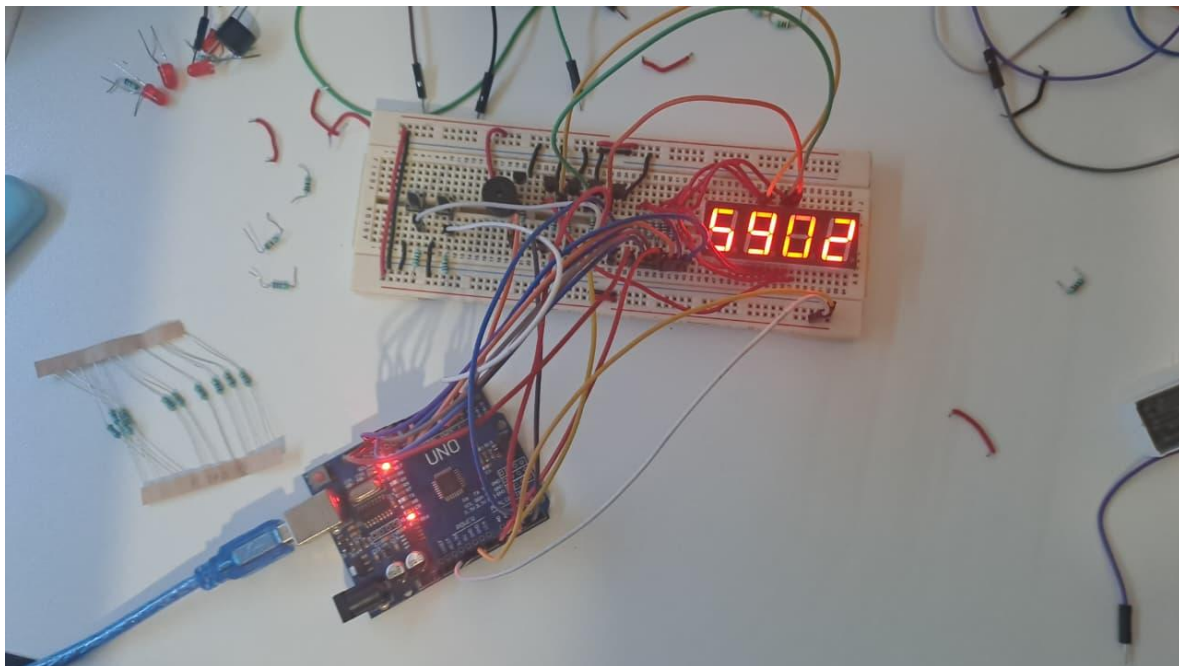
```
// ISR INT1 - Botón CONFIGURAR
ISR(INT1_vect)
{
    _delay_ms(50); // Antirrebote simple
    int1_debounce_flag = 1;
}
```

9. FOTOGRAFÍA DEL PROTOTIPO

ESQUEMÁTICO EN TINKERCAD



PROTOTIPO EN FÍSICO



CONCLUSIÓN PERSONAL

Heidi Pamela Martínez Martínez

1952947

#6

A lo largo de esta práctica aprendí que programar microcontroladores requiere un pensamiento completamente diferente al desarrollo de software convencional: cada bit cuenta, cada ciclo de reloj importa, y la eficiencia no es opcional sino necesaria. Dominar la manipulación directa de registros, implementar máquinas de estados para control de flujo, configurar interrupciones para respuesta en tiempo real, y sincronizar múltiples periféricos (display multiplexado, timers, botones y buzzer) me enseñó el valor de la planificación meticulosa y la documentación exhaustiva.

La capacidad de depurar código sin debugger visual, de anticipar comportamientos basándome en datasheets, y de diseñar soluciones robustas ante limitaciones estrictas son habilidades transferibles a cualquier disciplina de ingeniería. Esta práctica no solo me dio competencia técnica en microcontroladores AVR y lenguaje C embebido.

Steven Neftalí Verdugo Figueroa

2132366

#60

Esta actividad fue una oportunidad para aplicar de manera práctica conceptos avanzados de programación y control de periféricos. Desarrollar un sistema basado en interrupciones externas y temporizadas me permitió observar cómo se optimiza el rendimiento del microcontrolador al no depender de rutinas de espera o verificación continua (polling). La lógica del cronómetro, gestionada a través de una máquina de estados, fue clave para coordinar las transiciones entre configuración, ejecución y alarma final de forma coherente y predecible, reflejando una implementación de calidad profesional.

Un reto importante fue entender la manera correcta de configurar y sincronizar los timers internos, de modo que la base de tiempo fuera estable y precisa. En la simulación se observó la relevancia de los intervalos de interrupción bien definidos, ya que incluso pequeños errores en la temporización podían generar desviaciones en el conteo. Además, trabajar con los transistores de control para el display multiplexado me ayudó a entender cómo la electrónica discreta apoya la gestión visual de datos, optimizando el uso de pines del microcontrolador y mejorando la claridad del circuito.

A nivel personal, este proyecto fue una experiencia muy enriquecedora. Pude conectar la teoría aprendida en clase con un caso práctico completo que involucra hardware, programación y simulación. Comprendí que la verdadera habilidad de un programador de sistemas embebidos no está solo en escribir código, sino en diseñar un sistema completo, previendo cómo cada decisión técnica afecta al rendimiento global. En conclusión, esta práctica me dio confianza para enfrentar desafíos de diseño más complejos y reforzó mi interés por el desarrollo de sistemas electrónicos inteligentes.

En conclusión, la actividad fundamental que se realizó fue exitosa al implementar un cronómetro regresivo demostrando la programación avanzada de microcontroladores.

El proyecto logró la transición de la programación secuencial al modelo basado en interrupciones, lo que permitió una gestión asíncrona y eficiente de los eventos. Se utilizó la Máquina de Estados Finita (MEF) para estructurar la lógica de configuración, conteo y alarma, asegurando un comportamiento robusto.

El uso de interrupciones por timer para el conteo de 1s y el display multiplexado (evitando delays) optimizó el rendimiento del sistema en tiempo real. Esta experiencia confirma que las interrupciones y la MEF son fundamentales para diseñar sistemas embebidos de alto rendimiento y respuesta inmediata.

Esta actividad me permitió integrar conocimientos de electrónica digital, programación estructurada y diseño de sistemas embebidos en un proyecto real. La implementación del cronómetro regresivo usando interrupciones y una máquina de estados finita me ayudó a comprender cómo un microcontrolador puede manejar múltiples procesos simultáneamente de manera ordenada. Pasar de un esquema secuencial a uno basado en interrupciones fue un cambio de paradigma importante, pues demuestra cómo los sistemas modernos logran eficiencia, respuesta inmediata y ahorro de recursos de CPU.

Uno de los aprendizajes más valiosos fue el manejo de timers y control de tiempo sin recurrir a retardos (delays). Esta metodología permitió mantener la precisión del conteo y la multiplexación del display sin afectar la capacidad de respuesta del sistema ante eventos externos. También fue fundamental la integración de componentes electrónicos como transistores y resistencias para controlar las salidas, lo que me brindó una visión más completa sobre cómo el hardware debe ser pensado en conjunto con la lógica del software para obtener resultados óptimos. En el aspecto personal, considero que este proyecto fortaleció mis habilidades de análisis, diseño lógico y trabajo colaborativo. Aprendí a estructurar un programa de forma modular, a interpretar diagramas electrónicos en software EDA y a validar el funcionamiento de cada componente antes de la integración final.

REFERENCIAS BIBLIOGRÁFICAS

SIEMENS. (2025). Librerías para la comunicación con controladores SIMATIC. Siemens.com.

<https://support.industry.siemens.com/cs/document/109780503/librer%C3%ADas-para-la-comunicaci%C3%B3n-con-controladores-simatic?dti=0&lc=es-VE>

Ingeniería y Tecnología. (2023). ¿Qué son las librerías en programación y para qué sirven? UNIR; Universidad Internacional de La Rioja.

<https://www.unir.net/revista/ingenieria/librerias-programacion/>

Costa, D. (2024). Comprender e implementar el “Debouncing” para llamadas a la API en React. Apidog.com; Apidog Blog. <https://apidog.com/es/blog/understanding-and-implementing-debouncing-for-api-calls-in-react-4/>

TECmikro. (2025). Interrupciones de los microcontroladores PIC. TECmikro Ecuador. <https://tecmikro.com/content/68-interrupciones-microcontroladores-pic>

Jobit, J. (2022). ESP32 Timers & Timer Interrupts. Circuitdigest.com. <https://circuitdigest.com/microcontroller-projects/esp32-timers-and-timer-interrupts>

García, J. (2018). Ventajas y desventajas de las máquinas de estados finitos: cajas de interruptores, punteros C/C++ y tablas de búsqueda (Parte II). Ubidots Blog. https://es.ubidots.com/blog/advantages_and_disadvantages_of_finite_state_machines/

GCFGlobal. (2025). Secuencias, condicionales y ciclos. GCFGlobal.org. <https://edu.gcfglobal.org/es/conceptos-basicos-de-programacion/secuencias-condicionales-y-ciclos/1/>